PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
// Define the structure for a BST node
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
// Function to create a new node
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
// Function to insert a value into the BST
struct Node* insert(struct Node* root, int value) {
    // If the tree/subtree is empty, return a new node
    if (root == NULL) {
        return createNode(value);
    }

    // Otherwise, recur down the tree
    if (value < root->data) {
        root->left = insert(root->left, value);
    } else if (value > root->data) {
        root->right = insert(root->right, value);
    } else {
        printf("Duplicate value %d ignored.\n", value);
    }
    return root;
}

// Function to search for a value in the BST
struct Node* search(struct Node* root, int key) {
    // Base Cases: root is null or key is present at root
    if (root == NULL || root->data == key) {
        return root;
    }
```

```
    // Key is smaller than root's data
    if (key < root->data) {
        return search(root->left, key);
    }

    // Key is greater than root's data
    return search(root->right, key);
}

// Inorder traversal (Left, Root, Right) - Prints in sorted order
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

// Preorder traversal (Root, Left, Right)
void preorder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
```

```
// Inorder traversal (Left, Root, Right) - Prints in sorted order
void inorder(struct Node* root) {
   if (root != NULL) {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}

// Preorder traversal (Root, Left, Right)
void preorder(struct Node* root) {
   if (root != NULL) {
      printf("%d ", root->data);
      preorder(root->left);
      preorder(root->right);
   }
}

// Postorder traversal (Left, Right, Root)
void postorder(struct Node* root) {
   if (root != NULL) {
      postorder(root->left);
      postorder(root->right);
      printf("%d ", root->data);
   }
}

// Main function providing a menu-driven interface
int main() {
   struct Node* root = NULL;
   int choice, value, key;
   root = insert(root, 50);
   root = insert(root, 10);
   root = insert(root, 30);
   root = insert(root, 70);
   root = insert(root, 80);

            printf("Inorder Traversal: ");
```

```
            inorder(root);
            printf("\n");
            printf("Preorder Traversal: ");
            preorder(root);
            printf("\n");
            printf("Postorder Traversal: ");
            postorder(root);

            printf("\n Enter value to search: ");
            scanf("%d", &key);
            if (search(root, key) != NULL) {
               printf(" \n Element %d found in the BST.\n", key);
            } else {
               printf("\n Element %d NOT found in the BST.\n", key);
            }

   return 0;
}
```

OUTPUT:

Inorder Traversal: 10 30 50 70 80

Preorder Traversal: 50 10 30 70 80

Postorder Traversal: 30 10 80 70 50

Enter value to search: 30

Element 30 found in the BST.

Inorder Traversal: 10 30 50 70 80

Preorder Traversal: 50 10 30 70 80

Postorder Traversal: 30 10 80 70 50

Enter value to search: 55

Element 55 NOT found in the BST.